



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس مبانی رایانش توزیع شده
گزارش تمرین دوم

دانشجویان:

سارا گیتی ۸۱۰۱۰۱۵۰۰

پریا پاسه‌ورز ۸۱۰۱۰۱۳۹۳

فهرست مطالب

۶	فهرست تصاویر
۷	فهرست جداول
۸	۱ بخش اول: آشنایی با RPC و فناوری‌های مختلف آن
۸	۱-۱ شرح بخش
۸	۲-۱ تعریف RPC
۸	۳-۱ ایده اصلی RPC
۱۰	۴-۱ تفاوت فراخوانی محلی و فراخوانی راه دور
۱۰	۱-۴-۱ فراخوانی محلی (Local Procedure Call)
۱۰	۲-۴-۱ فراخوانی راه دور (Remote Procedure Call)
۱۱	۵-۱ نقش Server Stub و Client Stub
۱۱	۱-۵-۱ Client Stub
۱۱	۲-۵-۱ Server Stub
۱۲	۶-۱ معماری RPC
۱۳	۷-۱ مفهوم Serialization و Deserialization
۱۴	۱-۷-۱ مثال با Pickle Python
۱۴	۸-۱ مفهوم IDL (Interface Definition Language)
۱۵	۹-۱ خطاهای رایج در RPC
۱۶	۱-۹-۱ خطای عدم دسترسی به سرور
۱۷	۲-۹-۱ خطای از دست رفتن پیام (Message Loss)
۱۸	۳-۹-۱ ناسازگاری نوع داده‌ها
۱۸	۴-۹-۱ مشکل Pointerها و Referenceها
۱۹	۱۰-۱ انواع RPC
۱۹	۱-۱۰-۱ RPC همگام (Synchronous RPC)

۲۰	RPC ناهمگام (Asynchronous RPC)
۲۱	Deferred Synchronous RPC
۲۱	One-way RPC
۲۲	Multicast RPC
۲۲	مقایسه RPC با REST
۲۳	REST چیست؟
۲۳	تفاوت REST و RPC
۲۳	مقایسه چند فناوری رایج RPC
۲۳	gRPC
۲۵	JSON-RPC
۲۶	XML-RPC
۲۷	Apache Thrift
۲۷	جدول مقایسه فناوری‌های RPC
۲۸	پاسخ به پرسش‌های تحلیلی
	۱-۱۴ پرسش ۱: چرا RPC باعث می‌شود فراخوانی یک سرویس راه دور شبیه
۲۸	فراخوانی یک تابع محلی به نظر برسد؟
۲۸	۲-۱۴ پرسش ۲: چرا این شباهت می‌تواند خطرناک یا گمراه‌کننده باشد؟
	۳-۱۴ پرسش ۳: چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که
۲۹	در فراخوانی محلی وجود ندارد؟
۳۰	۴-۱۴ پرسش ۴: در چه شرایطی gRPC انتخاب بهتری از REST است؟
۳۰	۵-۱۴ پرسش ۵: در چه شرایطی REST انتخاب ساده‌تر یا مناسب‌تر است؟
۳۱	۱۵-۱ مثال واقعی RPC با gRPC در Go
۳۱	۱-۱۵ پروتو فایل
۳۲	۲-۱۵ سمت Server
۳۲	۳-۱۵ سمت Client
۳۳	۱۶-۱ نتیجه‌گیری نهایی

۱۷-۱ منابع ۳۳

۲ پیاده‌سازی سیستم توزیع شده چند ماشین مجازی ۳۴

۱-۲ معماری کل سیستم ۳۴

۱-۱-۲ ماشین مجازی اول (VM1): سرور وب اصلی ۳۴

۲-۱-۲ ماشین مجازی دوم (VM2): سرویس احراز هویت ۳۵

۳-۱-۲ ماشین مجازی سوم (VM3): سرویس ذخیره‌سازی فایل ۳۵

۴-۱-۲ ماشین مجازی چهارم (VM4): کارگزار توزیع پیام و مشترک ۳۵

۵-۱-۲ ماتریس اتصالات شبکه و پروتکل‌های ارتباطی ۳۶

۲-۲ نقش هر ماشین مجازی در سیستم توزیع شده ۳۶

۱-۲-۲ ماشین مجازی اول (VM1) - لایه ارائه و دیسچر اصلی (Web Server)

۳۶ (/ Gateway

۲-۲-۲ ماشین مجازی دوم (VM2) - لایه کنترل دسترسی (Backend Authentication) ۳۷

۳-۲-۲ ماشین مجازی سوم (VM3) - لایه ذخیره‌سازی شیء و مدیریت

۳۷ (File Server Asset) دارایی‌ها

۴-۲-۲ ماشین مجازی چهارم (VM4) - لایه مدیریت رویداد و پایش ترافیک

۳۸ (Pub/Sub Message Broker) ناهمگام

۳-۲ آدرس آی‌پی هر ماشین مجازی ۳۸

۴-۲ فناوری انتخاب شده برای RPC ۳۹

۱-۴-۲ فناوری RPC-JSON برای سرویس احراز هویت (VM1 به VM2) ۴۰

۲-۴-۲ فناوری RPC-Go برای سرویس ذخیره‌سازی فایل (VM1 به VM3) ۴۰

۳-۴-۲ مقایسه فنی پروتکل‌های RPC به کاررفته ۴۱

۵-۲ توضیح رویه‌های پیاده‌سازی شده ۴۱

۱-۵-۲ ۱. رویه احراز هویت در سرویس کنترل دسترسی (VM2) ۴۲

۲-۵-۲ ۲. رویه واکنشی دارایی‌ها در سرویس ذخیره‌سازی فایل (VM3) ۴۲

۳-۵-۲ ۳. ماتریس ساختار امضای رویه‌های از راه دور ۴۲

۴۲	۶-۲ توضیح نحوه دریافت فایل از VM3
۴۴	۷-۲ توضیح سازوکار الگوی انتشار/اشتراک
۴۵	۸-۲ توضیح روش پایش حافظه
۴۷	۹-۲ سناریوی تست و مستندسازی خروجی‌ها
۴۷	۲-۹-۱ گام ۱: نمایش آدرس IP ماشین‌های مجازی
۴۸	۲-۹-۲ گام ۲: اجرای سرویس احراز هویت در VM2
۴۸	۲-۹-۳ گام ۳: اجرای سرویس فایل در VM3
۴۸	۲-۹-۴ گام ۴: اجرای سرویس وب در VM1
۴۹	۲-۹-۵ گام ۵: اجرای کارگزار پیام (Broker) در VM4
۵۰	۲-۹-۶ گام ۶: باز کردن صفحه وب از سیستم میزبان
۵۰	۲-۹-۷ گام ۷: تلاش برای ورود با اطلاعات نادرست
۵۰	۲-۹-۸ گام ۸: مشاهده پیام خطای ورود
۵۲	۲-۹-۹ گام ۹: ارسال مشخصات معتبر و احراز هویت موفق
۵۲	۲-۹-۱۰ گام ۱۰: بازیابی خودکار دارایی‌های پویا از سرور فایل (VM3)
۵۳	۲-۹-۱۱ گام ۱۱: بارگذاری کامل داشبورد توزیع‌شده
۵۴	۲-۹-۱۲ گام ۱۲: اجرای مشترک (Subscriber) در VM4
۵۴	۲-۹-۱۳ گام ۱۳: شبیه‌سازی تخصیص حافظه و پایش وضعیت زیر مرز آستانه
۵۴	۲-۹-۱۴ گام ۱۴: شبیه‌سازی نشت حافظه و انتشار هشدار بحرانی بالادستی
۵۹	۲-۱۰ مشکلات و محدودیت‌های پیاده‌سازی

فهرست تصاویر

۱	بارگذاری موفقیت‌آمیز لایه واسط کاربری وب از طریق سیستم میزبان	۵۰
۲	مقداردهی فرم ورود با اطلاعات نامعتبر جهت تست خطای سیستم	۵۱
۳	نمایش پیام خطای عدم دسترسی (Access Denied) در مرورگر کلاینت	۵۱
۴	وارد کردن نام کاربری معتبر alice جهت ورود به سامانه	۵۲
۵	نمای داشبورد سیستم پس از ورود موفقیت‌آمیز و بارگذاری پویای تصویر کهکشان	۵۳
۶	شبیه‌سازی تخصیص حجم حافظه در لایه سرور وب (VM1) و پایش وضعیت	
	عادی زیر مرز ۳۰۰ مگابایت	۵۵
۷	دریافت و ثبت موفقیت‌آمیز فریم‌های تلمتری وضعیت پایدار توسط سرویس	
	Subscriber روی ماشین چهارم (VM4)	۵۵
۸	نمای واسط کاربری داشبورد پس از تحریک ابزار شبیه‌سازی نشت حافظه و عبور	
	از مرز ۳۰۰ مگابایت	۵۶
۹	ثبت لاگ فرآیند تخصیص پله‌پله حافظه و فعال شدن آستانه بحرانی در ترمینال	
	سرور وب (VM1)	۵۶

فهرست جداول

۱	مقایسه فراخوانی محلی و راه دور	۲۴
۲	مقایسه REST و RPC	۲۴
۳	جدول مقایسه فناوری‌های RPC	۲۷
۴	مشخصات اتصالات شبکه و پروتکل‌های میان ماشین‌های مجازی	۳۶
۵	نگاشت آدرس‌های آی‌پی و سرویس‌های گره‌های شبکه توزیع‌شده	۴۰
۶	مقایسه ویژگی‌های فنی زیرسیستم‌های RPC در کل شبکه	۴۱
۷	مشخصات خلاصه‌شده رویه‌ها و پورت‌های RPC سیستم	۴۳
۸	مشخصات زیرسیستم جریان رویدادها در الگوی Pattern Pub/Sub	۴۵
۹	پارامترهای فنی تنظیم‌شده برای زیرسیستم پایش حافظه در VM1	۴۷

۱ بخش اول: آشنایی با RPC و فناوری‌های مختلف آن

۱-۱ شرح بخش

در این بخش، یک گزارش کامل درباره RPC تهیه شده است. این گزارش دقیق، فنی و همراه با مقایسه فناوری‌های مختلف RPC است.

۲-۱ تعریف RPC

RPC مخفف Remote Procedure Call و به معنی فراخوانی رویه یا تابع از راه دور است. RPC یک مدل ارتباطی در سیستم‌های توزیع‌شده است که به یک برنامه اجازه می‌دهد تابعی را که روی یک کامپیوتر دیگر قرار دارد، همانند یک تابع محلی فراخوانی کند. به‌صورت ساده، در RPC برنامه کلاینت (Client) تابعی را فراخوانی می‌کند، اما اجرای واقعی تابع در یک ماشین دیگر (Server) صورت می‌گیرد. با این حال، این تفاوت برای برنامه‌نویس تا حد زیادی پنهان می‌شود.

در کل RPC را به‌عنوان مکانیزمی معرفی می‌شود که هدف آن پنهان‌سازی جزئیات ارسال و دریافت پیام‌های شبکه‌ای است. به جای آنکه برنامه‌نویس مستقیماً از توابع `send()` و `receive()` استفاده کند، تنها یک تابع را فراخوانی می‌کند؛ در حالی که سیستم RPC تمام جزئیات ارتباط را مدیریت می‌کند.

تعریف ۱.۱. RPC روشی است که در آن برنامه‌نویس می‌تواند تابعی را که روی سیستم دیگری اجرا می‌شود، همانند یک تابع محلی فراخوانی کند، بدون آنکه درگیر جزئیات پیچیده ارتباط شبکه‌ای شود.

۳-۱ ایده اصلی RPC

ایده اصلی RPC بر پایه ایجاد شفافیت در فراخوانی توابع از راه دور شکل گرفته است.

در حالت عادی، اگر بخواهیم دو برنامه روی دو ماشین مختلف با یکدیگر ارتباط برقرار کنند، باید:

۱. پیام مناسب ساخته شود.
۲. داده‌ها قالب‌بندی شوند.
۳. پیام از طریق شبکه ارسال شود.
۴. سمت مقابل پیام را دریافت کند.
۵. داده‌ها استخراج شوند.
۶. عملیات موردنظر اجرا شود.
۷. پاسخ مجدداً ارسال گردد.

این فرایند پیچیده و مستعد خطاست. RPC این پیچیدگی را مخفی می‌کند. در RPC، یک فراخوانی محلی تابع در سمت کلاینت به مجموعه‌ای از عملیات شبکه‌ای تبدیل می‌شود که در نهایت باعث اجرای همان تابع در سمت سرور می‌گردد، بدون آنکه کلاینت از این جزئیات آگاه باشد.

مثال ۲.۱. به جای نوشتن کد پیچیده شبکه:

```
1 send("GET_BALANCE", user_id)
2 response = receive()
```

برنامه‌نویس تنها می‌نویسد:

```
1 balance = getBalance(user_id)
```

در ظاهر این یک فراخوانی تابع عادی است؛ اما در پشت صحنه، سیستم RPC درخواست را به سرور ارسال کرده، تابع را اجرا می‌کند و نتیجه را بازمی‌گرداند.

۴-۱ تفاوت فراخوانی محلی و فراخوانی راه دور

یکی از موضوعات مهم در درک RPC، فهم تفاوت میان فراخوانی محلی و فراخوانی راه دور است.

۱-۴-۱ فراخوانی محلی (Local Procedure Call)

در فراخوانی محلی، تابع موردنظر در همان فضای حافظه و روی همان سیستم اجرا می‌شود.

```
1 sum = add(5, 7)
```

در اینجا:

- تابع روی همان سیستم اجرا می‌شود.
- دسترسی مستقیم به حافظه وجود دارد.
- هزینه زمانی کم است.
- نیازی به شبکه نیست.

۲-۴-۱ فراخوانی راه دور (Remote Procedure Call)

در RPC، تابع روی سیستم دیگری اجرا می‌شود.

```
1 user = getUserInfo(15)
```

ممکن است این تابع در واقع روی یک سرور در کشور دیگری اجرا شود.
در پشت صحنه:

۱. پارامترها بسته‌بندی می‌شوند.
۲. پیام از طریق شبکه ارسال می‌شود.
۳. سرور تابع را اجرا می‌کند.

۴. نتیجه بازگردانده می‌شود.

اما برای برنامه‌نویس همه چیز شبیه یک فراخوانی ساده به نظر می‌رسد. یکی از نکات مهم این است که RPC تلاش می‌کند تجربه فراخوانی محلی را شبیه‌سازی کند، اما هرگز دقیقاً همان نیست؛ زیرا محدودیت‌های شبکه، تأخیر، خرابی سیستم و ناسازگاری ماشین‌ها همواره وجود دارند.

۵-۱ نقش Server Stub و Client Stub

یکی از مهم‌ترین مفاهیم RPC، Stub است.

۱-۵-۱ Client Stub

Client Stub واسطه‌ای میان برنامه کلاینت و شبکه است. وظایف آن:

- دریافت فراخوانی تابع
 - بسته‌بندی پارامترها (Marshaling)
 - ارسال درخواست
 - انتظار برای پاسخ
 - بازگرداندن نتیجه
- Client Stub وظیفه دارد فراخوانی عادی تابع را به یک پیام شبکه‌ای تبدیل کند.

۲-۵-۱ Server Stub

Server Stub نقش معکوس دارد. وظایف:

- دریافت پیام
 - استخراج پارامترها (Unmarshaling)
 - فراخوانی تابع واقعی
 - آماده‌سازی پاسخ
- Server Stub درخواست شبکه‌ای را به یک فراخوانی محلی تابع تبدیل می‌کند؛ به‌طوری‌که از دید تابع سمت سرور، انگار یک فراخوانی عادی انجام شده است.

۶-۱ معماری RPC

معماری RPC معمولاً شامل اجزای زیر است:

Client •

Client Stub •

Network •

Server Stub •

Server •

روند اجرای RPC به‌صورت زیر است:

۱. کلاینت تابع را فراخوانی می‌کند.
۲. Client Stub پیام را می‌سازد.
۳. سیستم‌عامل پیام را از طریق شبکه ارسال می‌کند.
۴. پیام به Server Stub تحویل داده می‌شود.
۵. پارامترها استخراج می‌شوند.

۶. تابع واقعی اجرا می‌شود.

۷. نتیجه بسته‌بندی می‌شود.

۸. پاسخ به کلاینت بازگردانده می‌شود.

۹. Client Stub پاسخ را دریافت می‌کند.

۱۰. نتیجه به برنامه تحویل داده می‌شود.

۷-۱ مفهوم Serialization و Deserialization

یکی از مهم‌ترین مشکلات RPC انتقال داده بین ماشین‌های مختلف است. ماشین‌ها ممکن است:

- ساختار حافظه متفاوت داشته باشند

- ترتیب بایت متفاوت (Endianness) داشته باشند

- قالب ذخیره‌سازی متفاوتی داشته باشند

برای حل این مشکل از **Marshaling** و **Unmarshaling** استفاده می‌شود.

تعریف ۳.۱. Serialization (یا Marshaling): تبدیل داده به فرمت قابل انتقال.

مثال ۴.۱. تبدیل داده به JSON:

```
1 {  
2   "id": 10,  
3   "name": "Ali"  
4 }
```

تعریف ۵.۱. Deserialization (یا Unmarshaling): تبدیل داده دریافتی به شیء قابل استفاده.

۱-۷-۱ مثال با Pickle Python

نمونه‌ای از RPC در Python که از کتابخانه pickle برای Serialization استفاده می‌کند.

نمونه کد:

```
1 msgsnd = pickle.dumps(msglst)
2 retval = pickle.loads(msgrcv[1])
```

در اینجا:

- dumps() داده را serialize می‌کند.

- loads() داده را deserialize می‌کند.

۸-۱ مفهوم IDL (Interface Definition Language)

یکی از مشکلات RPC آن است که Client و Server باید دقیقاً بدانند:

- چه تابعی وجود دارد

- پارامترها چیست

- نوع داده‌ها چیست

برای این کار از IDL استفاده می‌شود.

IDL مخفف **Language Definition Interface** است.

IDL به توسعه‌دهنده اجازه می‌دهد رابط‌های RPC را تعریف کند و سپس سیستم به صورت

خودکار Client Stub و Server Stub را تولید نماید.

مثال ۶.۱. نمونه در gRPC:

```
1 syntax = "proto3";
2
3 service UserService {
```

```
4   rpc GetUser(UserRequest) returns (UserResponse);
5 }
6
7 message UserRequest {
8     int32 id = 1;
9 }
10
11 message UserResponse {
12     int32 id = 1;
13     string name = 2;
14 }
```

در اینجا:

• GetUser تابع راه دور است.

• UserRequest ورودی است.

• UserResponse خروجی است.

سپس gRPC به صورت خودکار کد Client و Server را تولید می‌کند.

۹-۱ خطاهای رایج در RPC

اگرچه RPC تلاش می‌کند فراخوانی تابع از راه دور را مشابه فراخوانی محلی جلوه دهد، اما در عمل تفاوت‌های مهمی میان این دو وجود دارد. یکی از اساسی‌ترین محدودیت‌های RPC این است که برخلاف فراخوانی محلی، ارتباط از طریق شبکه انجام می‌شود و شبکه ذاتاً غیرقابل اعتمادتر، کندتر و مستعد خطاست. در فراخوانی محلی، معمولاً فرض می‌شود که:

- حافظه در دسترس است،

- تابع وجود دارد،

- سیستم فعال است،

- و ارتباط تقریباً آنی است.

اما در RPC این فرض‌ها همیشه برقرار نیستند.

در کل RPC هرگز کاملاً معادل Local Procedure Call نیست، زیرا در محیط توزیع‌شده عواملی مانند خرابی شبکه، قطع ارتباط، تأخیر و ناسازگاری سیستم‌ها وجود دارند.

۱-۹-۱ خطای عدم دسترسی به سرور

یکی از رایج‌ترین مشکلات RPC زمانی رخ می‌دهد که سرور در دسترس نباشد.

برای مثال:

- سرور خاموش شده باشد،

- شبکه قطع شده باشد،

- یا سرویس RPC کرش کرده باشد.

در چنین شرایطی، کلاینت درخواست را ارسال می‌کند اما پاسخی دریافت نمی‌شود.

مثال ۷.۱. فرض کنید برنامه‌ای داریم:

```
1 user = getUser(15)
```

اگر سرور خاموش باشد، چه اتفاقی می‌افتد؟

در Local Call چنین مسئله‌ای تقریباً وجود ندارد، اما در RPC باید:

- Timeout تعریف شود،

- Retry mechanism وجود داشته باشد،

• Exception handling مناسب انجام شود.

```
1 try:
2     user = client.get_user(15)
3 except TimeoutError:
4     print("Server unavailable")
```

۱-۹-۲ خطای از دست رفتن پیام (Message Loss)

در شبکه ممکن است پیام:

• گم شود،

• دیر برسد،

• یا چند بار ارسال شود.

این موضوع در RPC بسیار مهم است، زیرا کلاینت ممکن است نداند آیا درخواست اجرا شده یا خیر.

مثال ۸.۱. نمونه Message Loss:

```
1 transferMoney(1000)
```

اگر پاسخ دریافت نشود، آیا پول منتقل شده است؟

سه حالت وجود دارد:

۱. درخواست هرگز نرسیده است.

۲. درخواست اجرا شده اما پاسخ گم شده است.

۳. سرور در میانه پردازش از کار افتاده است.

به همین دلیل RPC Framework ها سیاست‌هایی مانند:

• At-most-once

• At-least-once

• Exactly-once (تقریباً غیرممکن در عمل)

را پیاده‌سازی می‌کنند.

۳-۹-۱ ناسازگاری نوع داده‌ها

ماشین‌های مختلف ممکن است:

• معماری متفاوت داشته باشند،

• Endianness متفاوت داشته باشند،

• قالب ذخیره‌سازی متفاوتی برای اعداد داشته باشند.

با توجه به مسئله **Endian Big** و **Endian Little**، سیستم RPC باید داده‌ها را به فرمتی مستقل از ماشین تبدیل کند. به همین دلیل Serialization ضروری است.

۴-۹-۱ مشکل Pointerها و Referenceها

یکی از دشوارترین مسائل RPC انتقال Pointer است.

در کل Pointer فقط در فضای حافظه همان فرایند معنا دارد. بنابراین نمی‌توان آدرس حافظه یک سیستم را مستقیماً به سیستم دیگر فرستاد.

مثال ۹.۱. نمونه مشکل Pointer:

```
۱ update(&user)
```

در سیستم محلی مشکلی وجود ندارد، اما در RPC آدرس حافظه user روی سرور بی‌معنا خواهد بود.

راه‌حل معمول:

• Copy-by-value

• تبدیل Object به داده قابل انتقال

۱-۱۰ انواع RPC

چند مدل مختلف از RPC را معرفی می‌شود که بسته به نیاز سیستم استفاده می‌شوند. مهم‌ترین آن‌ها عبارت‌اند از:

۱. Synchronous RPC

۲. Asynchronous RPC

۳. Deferred Synchronous RPC

۴. One-way RPC

۵. Multicast RPC

۱-۱۰-۱ RPC همگام (Synchronous RPC)

این مدل، رایج‌ترین شکل RPC است.
در این حالت:

• کلاینت درخواست را ارسال می‌کند.

• منتظر پاسخ می‌ماند.

• تا زمان دریافت پاسخ، اجرای برنامه متوقف می‌شود.

شکل کلی:

Client ---- Request ----> Server

Client <--- Response ---- Server

در این مدل رفتار RPC بسیار شبیه فراخوانی عادی تابع است.

مثال ۱۰.۱. نمونه RPC:

```
1 result = getUser(10)
2 print(result)
```

برنامه تا بازگشت نتیجه متوقف می‌شود.

مزایا:

- ساده
- قابل فهم
- مشابه Local Call

معایب:

- Blocking
 - کاهش کارایی در سیستم‌های بزرگ
- این مدل را رفتار سنتی Request-Reply است.

۲-۱۰-۱ RPC ناهمگام (Asynchronous RPC)

در این مدل کلاینت پس از ارسال درخواست متوقف نمی‌شود. سرور تنها تأیید دریافت را ارسال می‌کند و پردازش ادامه می‌یابد. ساختار:

```
Client ---- Request ----> Server
Client <---- ACK ----- Server
Client continues...
```

مزیت:

- افزایش کارایی

- جلوگیری از Blocking

این روش زمانی مفید است که پاسخ فوری لازم نباشد یا چندین RPC به صورت موازی اجرا شوند.

۳-۱۰-۱ Deferred Synchronous RPC

در این مدل:

۱. کلاینت درخواست را ارسال می‌کند.

۲. تأیید دریافت می‌شود.

۳. کلاینت به کار خود ادامه می‌دهد.

۴. نتیجه بعداً از طریق Callback بازگردانده می‌شود.

این روش را برای زمانی مناسب است که کلاینت بخواهد هم‌زمان با چند سرور ارتباط برقرار کند.

۴-۱۰-۱ One-way RPC

در این مدل:

- کلاینت فقط درخواست را ارسال می‌کند.

- منتظر هیچ پاسخی نمی‌ماند.

مثال ۱۱.۱. نمونه درخواست One-way RPC:

```
logActivity(user_id)
```

مزایا:

- سریع
- مناسب Logging

معایب:

- عدم اطمینان از اجرای موفق

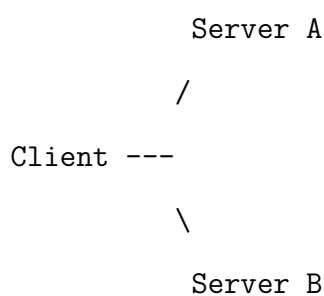
۵-۱۰-۱ Multicast RPC

مفهوم **RPC Multicast** یعنی، جایی که یک درخواست هم‌زمان به چند سرور ارسال می‌شود.

مثال کاربرد:

- Replication
- Fault tolerance
- Parallel processing

ساختار:



۱۱-۱ مقایسه REST با RPC

یکی از مهم‌ترین موضوعات در معماری سیستم‌های توزیع‌شده، مقایسه REST با RPC است.

۱-۱۱-۱ REST چیست؟

REST یک سبک معماری برای طراحی API‌های وب است که معمولاً از HTTP استفاده می‌کند.
نمونه:

GET /users/15

۲-۱۱-۱ تفاوت REST و RPC

تحلیل:

RPC معمولاً:

- سریع‌تر است،
- مناسب سرویس‌های داخلی است،
- در Microservice‌ها رایج‌تر است.

REST:

- ساده‌تر است،
- Debug آسان‌تری دارد،
- برای API عمومی بهتر است.

۱۲-۱ مقایسه چند فناوری رایج RPC

۱-۱۲-۱ gRPC

gRPC یکی از مدرن‌ترین Framework‌های RPC است که توسط Google توسعه یافته است.
ویژگی‌ها:

جدول ۱: مقایسه فراخوانی محلی و راه دور

ویژگی	فراخوانی محلی	فراخوانی راه دور
محل اجرا	همان سیستم	سیستم راه دور
نیاز به شبکه	ندارد	دارد
سرعت	بالا	کمتر
احتمال خطا	کم	بیشتر
Serialization	نیاز ندارد	نیاز دارد
پیچیدگی ارتباط	کم	زیاد

جدول ۲: مقایسه *RPC* و *REST*

ویژگی	<i>RPC</i>	<i>REST</i>
مدل	تابع محور	منبع محور
کارایی	بالا تر	کمتر
خوانایی	کمتر	بیشتر
سادگی <i>Debug</i>	سخت تر	آسان تر
مناسب <i>Microservice</i>	بسیار مناسب	مناسب
پروتکل	<i>TCP</i> ، <i>gRPC</i>	<i>HTTP</i>

- استفاده از HTTP/2
- سرعت بالا
- پشتیبانی از Streaming
- استفاده از Protocol Buffers

مثال ۱۲.۱. نمونه IDL:

```
1 service UserService {  
2     rpc GetUser(UserRequest) returns (UserResponse);  
3 }
```

مزایا:

- بسیار سریع
 - مناسب Microservice
 - Typed interface
- معایب:**
- Debug سخت‌تر
 - خوانایی کمتر نسبت به REST

۲-۱۲-۱ JSON-RPC

RPC مبتنی بر JSON است.

مثال ۱۳.۱. نمونه JSON:

```
1 {  
2   "jsonrpc": "2.0",  
3   "method": "subtract",  
4   "params": [42,23],  
5   "id": 1  
6 }
```

مزایا:

- ساده
- خوانا

معایب:

- کندتر از gRPC

۱-۱۲-۳ XML-RPC

یکی از قدیمی‌ترین فناوری‌های RPC است.

مثال ۱۴.۱. نمونه XML-RPC:

```
1 <methodCall>  
2   <methodName>getUser</methodName>  
3 </methodCall>
```

مزایا:

- استاندارد قدیمی و پایدار

معایب:

- حجیم و کند

۴-۱۲-۱ Apache Thrift

فریمورک RPC توسعه داده شده توسط Facebook.

ویژگی‌ها:

• Multi-language

• High performance

• Code generation

۱۳-۱ جدول مقایسه فناوری‌های RPC

جدول ۳: جدول مقایسه فناوری‌های RPC

فناوری	فرمت داده	IDL	موارد استفاده رایج	Streaming
gRPC	Protocol Buffers (باینری)	بله	Microservices، سیستم‌های پرتراфик	بله (دو طرفه)
JSON-RPC	JSON (متنی)	خیر/اختیاری	API‌های ساده، پروژه‌های کوچک	خیر
XML-RPC	XML (متنی)	خیر	سیستم‌های قدیمی (Legacy)	خیر
Apache Thrift	باینری (قابل تنظیم)	بله	سیستم‌های ناهمگون، نیاز به انعطاف بالا	محدود

۱۴-۱ پاسخ به پرسش‌های تحلیلی

در این بخش، به پرسش‌های تحلیلی مطرح‌شده در تمرین پاسخ داده می‌شود. هدف این قسمت صرفاً ارائهٔ پاسخ کوتاه نیست، بلکه تحلیل فنی و استدلال مهندسی در مورد انتخاب فناوری RPC مناسب در سناریوهای مختلف است.

۱-۱۴-۱ پرسش ۱: چرا RPC باعث می‌شود فراخوانی یک تابع محلی به نظر برسد؟

RPC با استفاده از مکانیزم **Stub** این شبیه‌سازی را انجام می‌دهد. **Client Stub** همان توابع محلی را در اختیار برنامه‌نویس قرار می‌دهد. وقتی برنامه‌نویس تابعی مانند `GetUser(15)` را فراخوانی می‌کند، در پشت صحنه **Stub** کارهای زیر را انجام می‌دهد:

۱. پارامترها را **Serialize** می‌کند.

۲. درخواست را از طریق شبکه ارسال می‌کند.

۳. منتظر پاسخ می‌ماند.

۴. پاسخ را **Deserialize** کرده و نتیجه را برمی‌گرداند.

از دید برنامه‌نویس، فراخوانی `GetUser(15)` کاملاً شبیه یک تابع محلی است. به این ویژگی **Location Transparency** یا شفافیت مکان می‌گویند.

۱-۱۴-۲ پرسش ۲: چرا این شباهت می‌تواند خطرناک یا گمراه‌کننده باشد؟

این شباهت خطرناک است زیرا فراخوانی محلی و راه دور ذاتاً تفاوت‌های بنیادی دارند:

۱. **زمان اجرا:** فراخوانی محلی در نانوثانیه، اما فراخوانی راه دور در میلی‌ثانیه تا ثانیه (به خاطر تأخیر شبکه).

۲. **خطاها:** در فراخوانی محلی خطاهایی مانند Timeout، قطع شدن شبکه یا خرابی سرور معنا ندارند.

۳. **اشتراک حافظه:** فراخوانی محلی به حافظه مشترک دسترسی دارد (اشاره‌گرها کار می‌کنند)، اما در RPC نمی‌توان اشاره‌گر ارسال کرد.

خطر اصلی: برنامه‌نویس ممکن است فکر کند کار ساده است و برای خطاهای شبکه، Timeout و Partial Failure برنامه‌ای ننویسد. در نتیجه سیستم در شرایط واقعی توزیع‌شده به درستی کار نمی‌کند.

در کل RPC هرگز کاملاً معادل Local Procedure Call نیست. همچنان با توجه به جمله معروف “Network is not transparent”.

۱-۱۴-۳ پرسش ۳: چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که در فراخوانی محلی وجود ندارد؟

۱. **خطای عدم دسترسی به سرور:** سرور خاموش باشد، شبکه قطع باشد یا سرویس کرش کرده باشد.

۲. **Timeout:** سرور پاسخ ندهد (به دلیل شلوغی یا قفل شدن).

۳. **از دست رفتن پیام (Message Loss):** درخواست یا پاسخ در شبکه گم شود.

۴. **Partial Failure:** درخواست به سرور برسد و پردازش شروع شود، اما وسط کار سرور کرش کند. کلاینت نمی‌داند عملیات انجام شده یا نه.

۵. **ناسازگاری داده‌ها (Serialization Mismatch):** مثلاً سمت کلاینت یک فیلد جدید داشته باشد که سرور نمی‌شناسد.

۶. **مشکل Idempotency:** اگر درخواست دوباره فرستاده شود (به خاطر Timeout)، ممکن است عملیات دو بار انجام شود (مثل انتقال پول).

۷. **مشکل Pointerها:** اشاره‌گرها فقط در فضای حافظه همان فرایند معنا دارند.

۱-۱۴-۴ پرسش ۴: در چه شرایطی gRPC انتخاب بهتری از REST است؟

gRPC در شرایط زیر بهتر است:

۱. ارتباط داخلی Microservice ها: سرعت بالا و سربار کم برای ارتباط بین سرویس‌ها حیاتی است.
۲. نیاز به Streaming: gRPC از Streaming دو طرفه پشتیبانی می‌کند (مثل سیستم‌های چت یا مانیتورینگ لحظه‌ای).
۳. سیستم‌های چندزبانه (Polyglot): gRPC با تولید خودکار کد برای زبان‌های مختلف، هماهنگی را تضمین می‌کند.
۴. نیاز به کارایی بالا: فرمت باینری Protobuf نسبت به JSON حجم کمتری دارد و سریع‌تر است. در سیستم‌هایی که هزاران درخواست در ثانیه دارند، این تفاوت بسیار مهم است.
۵. معماری Contract-First: وقتی بخواهیم قرارداد سرویس را دقیقاً مشخص کنیم.
۶. استفاده از HTTP/2: قابلیت‌هایی مانند Multiplexing، Header Compression و Persistent Connections که تأخیر شبکه را کاهش می‌دهند.

۱-۱۴-۵ پرسش ۵: در چه شرایطی REST انتخاب ساده‌تر یا مناسب‌تر است؟

REST در شرایط زیر مناسب‌تر است:

۱. API عمومی (Public API): مرورگرها و کلاینت‌های مختلف به راحتی با JSON کار می‌کنند. gRPC نیازمند ابزارهای خاص است.
۲. پروژه‌های کوچک و سریع: نوشتن یک API با Flask یا Express بسیار سریع‌تر از تعریف فایل Protobuf و تولید کد است.
۳. نیاز به Caching: REST در سطح HTTP کش شدن را پشتیبانی می‌کند، gRPC خیر.

۴. سیستم‌های ساده CRUD: کارهایی که فقط شامل ایجاد، خواندن، به‌روزرسانی و حذف منابع هستند. REST با متدهای GET، POST، PUT، DELETE طبیعی‌تر و خواناتر است.

۵. دیباگ کردن: می‌توان با curl یا Postman مستقیماً درخواست فرستاد و پاسخ JSON را دید. gRPC به ابزار مخصوص (grpcurl) نیاز دارد.

۶. سازگاری بیشتر: REST تقریباً روی همه پلتفرم‌ها کار می‌کند، اما gRPC گاهی با Firewall، Proxy یا مرورگر مشکل دارد.

۱۵-۱ مثال واقعی RPC با gRPC در Go

با توجه به اینکه بخش عملی پروژه با Go انجام می‌شود، استفاده از مثال gRPC در Go می‌تواند کیفیت گزارش را افزایش دهد.

۱-۱۵-۱ فایل proto

```
1 syntax = "proto3";
2
3 package user;
4
5 service UserService {
6     rpc GetUser(UserRequest) returns (UserResponse);
7 }
8
9 message UserRequest {
10     int32 id = 1;
11 }
12
13 message UserResponse {
```

```
14     int32 id = 1;
15     string name = 2;
16 }
```

۲-۱۵-۱ سمت Server

```
1 func (s *server) GetUser(
2     ctx context.Context,
3     req *pb.UserRequest,
4 ) (*pb.UserResponse, error) {
5     return &pb.UserResponse{
6         Id:    req.Id,
7         Name:  "Ali",
8     }, nil
9 }
```

۳-۱۵-۱ سمت Client

```
1 resp, err := client.GetUser(
2     context.Background(),
3     &pb.UserRequest{Id: 15},
4 )
5 fmt.Println(resp.Name)
```

این مثال نشان می‌دهد که از دید برنامه‌نویس:

```
1 client.GetUser()
```

فقط یک تابع عادی است، در حالی که در پشت صحنه RPC اتفاق می‌افتد.

۱۶-۱ نتیجه‌گیری نهایی

RPC یکی از بنیادی‌ترین مفاهیم سیستم‌های توزیع‌شده است که هدف اصلی آن پنهان‌سازی پیچیدگی‌های ارتباط شبکه‌ای و فراهم‌کردن شفافیت دسترسی است. در RPC، برنامه‌نویس می‌تواند توابع راه دور را مشابه توابع محلی فراخوانی کند، در حالی که جزئیات ارتباط توسط Stubها، Serialization و پروتکل‌های ارتباطی مدیریت می‌شوند. اگرچه RPC تلاش می‌کند تفاوت میان فراخوانی محلی و راه دور را مخفی کند، اما محدودیت‌هایی مانند تأخیر شبکه، خرابی سیستم، ناسازگاری داده‌ها و مشکلات انتقال Referenceها همواره وجود دارند. به همین دلیل RPC هرگز جایگزین کامل Local Call نیست. در میان فناوری‌های موجود، gRPC به دلیل کارایی بالا، استفاده از HTTP/2، پشتیبانی از Pro-tocol Buffers و قابلیت تولید خودکار کد، گزینه‌ای بسیار مناسب برای معماری‌های Microservice است. در مقابل، فناوری‌هایی مانند REST یا JSON-RPC به دلیل سادگی و خوانایی، همچنان در بسیاری از پروژه‌ها کاربرد گسترده دارند. در نهایت، انتخاب فناوری مناسب RPC وابسته به نیازهای پروژه، مقیاس سیستم، نوع ارتباط، شرایط شبکه و میزان اهمیت کارایی و سادگی توسعه است.

۱۷-۱ منابع

۱. Edith Systems, Distributed Tanenbaum, S. Andrew Steen, van Maarten tion.

۲. <https://grpc.io/docs/>: gRPC Official Documentation

۳. <https://protobuf.dev/>: Protocol Buffers Documentation

۴. <https://thrift.apache.org/>: Apache Thrift Documentation

۵. <https://www.jsonrpc.org/specification>: JSON-RPC Specification

۲ پیاده‌سازی سیستم توزیع شده چند ماشین مجازی

۱-۲ معماری کل سیستم

سیستم توزیع‌شده پیاده‌سازی شده در این پروژه، از یک معماری مبتنی بر ریزسرویس‌های مستقل (Microservices) تشکیل شده است که به‌منظور افزایش امنیت، جداسازی وظایف (Separation of Concerns) و مقیاس‌پذیری، بر روی چهار ماشین مجازی (VM) مجزا مستقر شده‌اند. هدف اصلی این معماری، مدیریت درخواست‌های کاربران، احراز هویت امن و پایش لحظه‌ای معیارهای سلامت سیستم بدون ایجاد وابستگی شدید (Tight Coupling) میان اجزا است.

اجزای تشکیل‌دهنده این شبکه توزیع‌شده و نحوه تعامل آن‌ها بر اساس آخرین پیکربندی‌های عملیاتی به شرح زیر است:

۱-۱-۲ ماشین مجازی اول (VM1): سرور وب اصلی

این ماشین با آدرس آی‌پی 192.168.176.129 به عنوان دروازه ورود سیستم (Application Gateway) عمل می‌کند و تنها بخشی است که به صورت مستقیم با مرورگر کاربر در ارتباط است. وظایف کلیدی این گره عبارتند از:

- مدیریت مسیرهای سیستم (Routing Matrix) شامل پورتال ورود، داشبورد مدیریتی و زیرسیستم شبیه‌ساز نشت حافظه روی پورت ۸۰۸۰.
- ارتباط با سایر ماشین‌های مجازی از طریق فراخوانی رویه‌های از راه دور (RPC) جهت دریافت داده‌ها بدون ذخیره‌سازی محلی آن‌ها.
- اجرای یک رشته پس‌زمینه پایشگر (Watchdog Watcher) با استفاده از ابزار time.Ticker که هر ۲ ثانیه یک‌بار وضعیت حافظه مصرفی گره را از طریق runtime.ReadMemStats بررسی می‌کند. در صورت عبور حافظه از مرز ۳۰۰ مگابایت (به عنوان مثال رسیدن به مقدار ۳۱۰ مگابایت پس از تخصیص حافظه خطای شبیه‌سازی‌شده)، این پایشگر یک

هشدار ساختاریافته به کارگزار پیام ارسال می‌کند.

۲-۱-۲ ماشین مجازی دوم (VM2): سرویس احراز هویت

این زیرسیستم با آدرس آی‌پی 192.168.176.130 به صورت کاملاً مجزا عمل کرده و وظیفه تایید هویت کاربران را بر عهده دارد. سرور وب (VM1) هیچ‌گونه دسترسی مستقیمی به پایگاه داده کاربران ندارد و مشخصات اعتبارسنجی را از طریق پروتکل JSON-RPC به پورت ۸۰۸۲ این ماشین ارسال می‌کند. پاسخ تایید یا رد درخواست پس از بررسی فایل محلی users.json به گره اصلی بازگردانده می‌شود.

۳-۱-۲ ماشین مجازی سوم (VM3): سرویس ذخیره‌سازی فایل

این ماشین با آدرس آی‌پی 192.168.176.131 به عنوان مخزن اصلی اسناد عمل می‌کند. به دلیل رعایت اصول سیستم‌های توزیع‌شده، هیچ فایل تصویری روی سرور وب بارگذاری نشده است. پس از تایید دسترسی کاربر در داشبورد، سرور وب با استفاده از پروتکل بومی Go-RPC روی پورت ۸۰۸۳ به این ماشین متصل شده و داده‌های باینری تصاویر را به صورت جریان پیوسته از بایت‌ها (Byte Stream) دریافت کرده و به مرورگر کاربر تحویل می‌دهد.

۴-۱-۲ ماشین مجازی چهارم (VM4): کارگزار توزیع پیام و مشترک

این گره با آدرس آی‌پی 192.168.176.132 وظیفه مدیریت ناهمگام (Asynchronous) هشدارهای ترافیک و نشت حافظه سیستم را بر عهده دارد و شامل دو بخش اصلی است:

- **کارگزار پیام (Broker):** این برنامه به صورت همزمان دو پورت شبکه را مدیریت می‌کند. پورت ۸۰۸۴ وظیفه دریافت هشدارهای خرابی از سرور وب را از طریق پروتکل HTTP POST بر عهده دارد و پورت هشدارهای دریافتی را به صورت لحظه‌ای روی سوکت‌های فعال TCP برای ابزارهای ناظر ارسال می‌کند.
- **برنامه مشترک (Subscriber):** یک ابزار سبک در محیط ترمینال است که با دریافت پویای

آدرس آی‌پی کارگزار از طریق فلگ‌های زمان اجرا، به آن متصل شده و هشدارهای مربوط به عبور از آستانه مصرف حافظه را به صورت زنده چاپ می‌کند.

۲-۱-۵ ماتریس اتصالات شبکه و پروتکل‌های ارتباطی

جدول ۴ خلاصه‌ای از جریان‌های ارتباطی، شماره پورت‌ها و پروتکل‌های به‌کاررفته در میان اجزای مختلف معماری توزیع‌شده پروژه را بر اساس مقادیر نهایی و عملیاتی نودها نشان می‌دهد.

جدول ۴: مشخصات اتصالات شبکه و پروتکل‌های میان ماشین‌های مجازی

دستگاه مبدأ	آدرس آی‌پی مبدأ	دستگاه مقصد	شماره پورت مقصد	پروتکل ارتباطی
VM1	192.168.176.129	VM2	۸۰۸۲	RPC-JSON
VM1	192.168.176.129	VM3	۸۰۸۳	RPC-Go
VM1	192.168.176.129	VM4	۸۰۸۴	POST HTTP
Subscriber	192.168.176.132	VM4	۸۰۸۵	Socket TCP

۲-۲ نقش هر ماشین مجازی در سیستم توزیع‌شده

برای درک بهتر رفتار هماهنگ سیستم و جداسازی وظایف، نقش عملیاتی و لایه پردازشی هر یک از ماشین‌های مجازی به همراه زیرسیستم‌های در حال اجرای آن‌ها به تفکیک در این بخش تشریح شده است. این معماری به گونه‌ای طراحی شده است که مستقل از ماهیت و نوع داده‌ها عمل کرده و توانایی مدیریت، احراز هویت و استریم انواع فایل‌ها و دارایی‌های دیجیتال گوناگون را دارا می‌باشد:

۲-۲-۱ ماشین مجازی اول (VM1) - لایه ارائه و دیسپچر اصلی (Web Server / Gateway)

این ماشین با آدرس آی‌پی 192.168.176.129 به عنوان نقطه تماس اصلی با دنیای خارج و کاربران عمل می‌کند. نقش‌های کلیدی آن عبارتند از:

- **واسط کاربری و پردازش فرم‌ها:** میزبانی از صفحات HTML و استایل‌های CSS مربوط به صفحه ورود و داشبورد جامع مدیریتی سیستم جهت تعامل کاربران و نمایش فایل‌ها روی پورت ۸۰۸۰.

- **پروکسی و واسط ارتباطی (RPC Proxy):** این ماشین فاقد هرگونه داده حساس یا فایل‌های باینری محلی است؛ بنابراین درخواست‌های احراز هویت را به VM2 و درخواست‌های واکشی فایل را به VM3 هدایت می‌کند.

- **مکانیزم خودکار پایش (Telemetry Engine):** این ماشین وضعیت پویای حافظه تخصیص‌یافته روی هپ (Heap) را دائماً اندازه‌گیری کرده و در صورت عبور از مرز ۳۰۰ مگابایت، مسئولیت تولید سیگنال خطا و ارسال ناهمگام آن به کارگزار پیام را بر عهده دارد.

۲-۲-۲ ماشین مجازی دوم (VM2) - لایه کنترل دسترسی (Backend Authentication)

این ماشین نقش یک مرجع صدور گواهی و تایید هویت متمرکز را ایفا می‌کند و امنیت دسترسی به سیستم را تضمین می‌نماید:

- **ایزوله‌سازی داده‌های حساس:** نگهداری امن از فایل حساب‌های کاربری (users.json) به طوری که مخاطرات امنیتی یا نفوذ به سرور وب اصلی (VM1) منجر به افشای اطلاعات ورود کاربران نشود.

- **پاسخگویی به رویه‌های از راه دور:** این ماشین به عنوان یک سرور JSON-RPC روی پورتهای ۸۰۸۲ مستقر شده است و صرفاً درخواست‌های اعتبارسنجی را پردازش کرده و نتیجه را به صورت ساختاریافته بازمی‌گرداند.

۳-۲-۲ ماشین مجازی سوم (VM3) - لایه ذخیره‌سازی شیء و مدیریت دارایی‌ها (File Server Asset)

نقش این گره با آدرس آی‌پی 192.168.176.131، مدیریت انواع فایل‌های با حجم بالا و کاهش بار پردازشی و دیسک از روی سرور وب است:

• **مخزن جامع دارایی‌های دیجیتال (Digital Asset Repository):** ذخیره متمرکز انواع فایل‌ها، مستندات، تصاویر چندرسانه‌ای با رزولوشن بالا و پکت‌های داده در دایرکتوری محلی files/ یا images/. این زیرسیستم بستری همگانی برای ذخیره‌سازی اشیاء (Object Storage) فراهم می‌کند.

• **استریم بهینه تصاویر و فایل‌ها (Binary Byte Streaming):** اجرای سرور Go-RPC روی پورت ۸۰۸۳ جهت تبدیل فایل‌های درخواستی به جریان‌های باینری و ارسال مستقیم داده‌ها روی سوکت شبکه بدون نیاز به انکودینگ‌های متنی سنگین.

۴-۲-۴. ماشین مجازی چهارم (VM4) - لایه مدیریت رویداد و پایش ترافیک ناهمگام (Pub/Sub Message Broker)

این گره با آدرس آی‌پی 192.168.176.132 به عنوان هاب مدیریت خرابی و مانیتورینگ سیستم عمل کرده و بار پردازش هشدارهای سیستم را به طور کامل از زنجیره اصلی درخواست‌های کاربران خارج می‌کند:

• **جداسازی پردازش هشدارها (Asynchronous Alerting):** به جای اینکه وب‌سرور درگیر پردازش یا ذخیره‌سازی لاگ‌های خرابی شود، هشدارهای خود را به صورت ناهمگام به پورت ۸۰۸۴ این ماشین می‌فرستد.

• **توزیع توامان رویدادها (Event Distribution):** انتشار همزمان رویدادهای نشت حافظه دریافتی به کلاینت‌های ترمینال مشترک (Subscriber) روی پورت ۸۰۸۵ برای نظارت زنده مهندسان سیستم بر وضعیت سلامت سرورها.

۳-۲. آدرس آی‌پی هر ماشین مجازی

در این پروژه، به منظور شبیه‌سازی دقیق و گام‌به‌گام یک محیط شبکه توزیع‌شده واقعی، از یک زیرشبکه (Subnet) اختصاصی در محیط مجازی‌ساز استفاده شده است. تمامی ماشین‌های

مجازی به صورت استاتیک (Static IP) در این زیرشبکه پیکربندی شده‌اند تا تعاملات میان‌گره‌ای و فراخوانی رویه‌های از راه دور بدون تداخل و به صورت کاملاً پایدار برقرار شود. توزیع آدرس‌های آی‌پی و پورت‌های عملیاتی تخصیص‌یافته به سرویس‌های هر گره بر اساس پیکربندی نهایی سیستم به شرح زیر است:

• **ماشین مجازی اول (VM1):** این ماشین با آدرس آی‌پی 192.168.176.129 به عنوان سرور وب اصلی و دروازه سیستم عمل می‌کند و روی پورت ۸۰۸۰ به درخواست‌های ارسالی از مرورگر کاربران پاسخ می‌دهد.

• **ماشین مجازی دوم (VM2):** این ماشین با آدرس آی‌پی 192.168.176.130 به عنوان سرویس‌دهنده اختصاصی مدیریت کنترل دسترسی عمل کرده و روی پورت ۸۰۸۲ به درخواست‌های وب سرور پاسخ می‌دهد.

• **ماشین مجازی سوم (VM3):** این ماشین با آدرس آی‌پی 192.168.176.131 وظیفه میزبانی مخزن متمرکز فایل‌ها را بر عهده دارد و ارتباطات بک‌اند را روی پورت ۸۰۸۳ پذیرا می‌باشد.

• **ماشین مجازی چهارم (VM4):** این ماشین با آدرس آی‌پی 192.168.176.132 زیرسیستم ناهمگام مدیریت رویدادها را مدیریت می‌کند و پورت‌های ۸۰۸۴ (پروتکل متنی رویداد) و ۸۰۸۵ (سوکت توزیع پیام) را به خود اختصاص داده است.

جدول ۵ جزئیات دقیق نگاشت زیرشبکه، آدرس‌های آی‌پی و پورت‌های نهایی مورد استفاده در این پروژه را نشان می‌دهد.

۴-۲ فناوری انتخاب‌شده برای RPC

در این پروژه، برای برقراری ارتباط ساختاریافته میان ماشین‌های مجازی و فراخوانی توابع از راه دور، از فناوری RPC (Remote Procedure Call) استفاده شده است. از آنجا که نیازمندی‌های لایه کنترل دسترسی و احراز هویت (VM2) با زیرسیستم ذخیره‌سازی و انتقال شیء (VM3) کاملاً متفاوت است، دو پیاده‌سازی گوناگون از این فناوری انتخاب شد تا تعادلی بهینه میان امنیت، استانداردهای متنی و سرعت انتقال داده در شبکه ایجاد شود.

جدول ۵: نگاشت آدرس‌های آی‌پی و سرویس‌های گره‌های شبکه توزیع‌شده

شناسه ماشین	آدرس آی‌پی	پورت شبکه	نقش سرویس
VM1	۱۹۲/۱۶۸/۱۷۶/۱۲۹	۸۰۸۰	Server Web
VM2	۱۹۲/۱۶۸/۱۷۶/۱۳۰	۸۰۸۲	Backend Authentication
VM3	۱۹۲/۱۶۸/۱۷۶/۱۳۱	۸۰۸۳	Server File
VM4	۱۹۲/۱۶۸/۱۷۶/۱۳۲	۸۰۸۴/۸۰۸۵	Broker Message

۲-۴-۱ فناوری RPC-JSON برای سرویس احراز هویت (VM1 به VM2)

برای تبادل داده‌های مربوط به تایید هویت کاربران بین سرور وب اصلی و لایه بک‌اند، پروتکل RPC-JSON انتخاب شده است. دلایل این انتخاب فنی عبارتند از:

- **قالب متنی استاندارد (Text Format):** داده‌های اعتبارسنجی شامل نام کاربری و رمز عبور در قالب متنی و ساختاریافته JSON مبادله می‌شوند که خوانایی بالایی داشته و فرآیند پایش پکت‌ها و عیب‌یابی شبکه را تسهیل می‌کند.
- **استقلال از زبان (Language Independence):** معماری RPC-JSON یک استاندارد عمومی و مستقل از پلتفرم است. در صورتی که در آینده نیاز باشد سرویس احراز هویت VM2 با زبان یا فریم‌ورک دیگری بازنویسی شود، هیچ نیازی به تغییر در کدهای کلاینت در سرور وب (VM1) نخواهد بود.

۲-۴-۲ فناوری RPC-Go برای سرویس ذخیره‌سازی فایل (VM1 به VM3)

به منظور واکنشی و استریم دارایی‌های دیجیتال و فایل‌های حجیم از مخزن متمرکز به سرور وب اصلی، از بسته بومی net/rpc در زبان Go (تحت عنوان پروتکل RPC-Go) استفاده شده است. علل اتخاذ این تصمیم عبارتند از:

- **رمزگذاری باینری بهینه (Binary Encoding):** این پروتکل به صورت بومی از قالب Gob برای سریال‌سازی داده‌ها استفاده می‌کند. برخلاف پروتکل‌های متنی (مانند JSON) که

داده‌های باینری را به رشته‌های متنی سنگین تبدیل کرده و بار شبکه را افزایش می‌دهند، RPC-Go بایت‌های خام فایل را در قالب جریان‌های فشرده باینری (Byte Stream) و با کمترین سربار روی شبکه منتقل می‌کند.

• **کارایی و سرعت بالا (High Throughput):** فرآیند انکود و دکود در پروتکل بومی زبان Go به شدت سریع‌تر از پردازش‌های متنی است که این امر زمان پاسخ‌گویی و بارگذاری دارایی‌ها را در سمت کاربر به حداقل می‌رساند.

۳-۴-۲ مقایسه فنی پروتکل‌های RPC به‌کاررفته

جدول ۶ مقایسه‌ای جامع میان دو فناوری انتخابی برای معماری توزیع‌شده پروژه را بر اساس معیارهای عملیاتی نشان می‌دهد.

جدول ۶: مقایسه ویژگی‌های فنی زیرسیستم‌های RPC در کل شبکه

معیار مقایسه	سرویس احراز هویت (VM2)	سرویس ذخیره‌سازی فایل (VM3)
فناوری انتخاب‌شده	RPC-JSON	RPC-Go
شماره پورت اتصال	۸۰۸۲	۸۰۸۳
قالب سریال‌سازی	Serialization Text	Encoding Binary
نوع داده مبادلاتی	Data Structured	Stream Byte
تمرکز اصلی لایه	Interoperability and Safety	Performance and Speed

۵-۲ توضیح رویه‌های پیاده‌سازی شده

در این سیستم توزیع‌شده، تبادل داده‌ها و دستورات میان سرور وب اصلی (VM1) و سرویس‌های بک‌اند از طریق فراخوانی امضا‌های مشخصی از توابع از راه دور (RPC Procedures) انجام می‌شود. هر سرویس مجموعه‌ای از ساختارهای ورودی (Arguments) و ساختارهای خروجی (Reply) را تعریف می‌کند تا یکپارچگی داده‌ها در طول شبکه حفظ شود.

۲-۵-۱. رویه احراز هویت در سرویس کنترل دسترسی (VM2)

این رویه وظیفه بررسی و تایید اعتبارسنجی نشست‌های کاربران را بر عهده دارد:

• نام رویه: AuthService.Login

• مکانیزم عملکرد: کلاینت سرور وب، نام کاربری و رمز عبور دریافتی از فرم HTML را در قالب یک ساختار به این رویه ارسال می‌کند. سرور JSON-RPC مستقر در VM2 با جستجو در فایل متنی مقادیر را اعتبارسنجی کرده و نتیجه تایید یا رد را به همراه پیام مربوطه بازمی‌گرداند.

۲-۵-۲. رویه واکنشی دارایی‌ها در سرویس ذخیره‌سازی فایل (VM3)

این رویه وظیفه انتقال بهینه و بلادرنگ شیء‌ها و دارایی‌های دیجیتال را بر عهده دارد:

• نام رویه: FileService.FetchFile

• مکانیزم عملکرد: سرور وب اصلی پس از تایید هویت کاربر، نام فایل درخواستی را به این رویه ارسال می‌کند. سرور بومی Go-RPC در VM3 فایل متناظر را از دایرکتوری دارایی‌ها به صورت باینری لود کرده و آرایه بایت‌های خام آن را از طریق ساختار پاسخ، روی شبکه استریم می‌کند.

۲-۵-۳. ماتریس ساختار امضای رویه‌های از راه دور

جدول ۷ جزئیات ساختار ورودی، خروجی و پورت‌های نهایی تخصیص‌یافته به هر یک از رویه‌های پیاده‌سازی شده در معماری کل سیستم را نشان می‌دهد.

۲-۶. توضیح نحوه دریافت فایل از VM3

یکی از اهداف اصلی این کلود توزیع‌شده، جداسازی لایه ارایه از لایه ذخیره‌سازی داده‌ها است. به همین منظور، هیچ‌گونه رسانه یا دارایی دیجیتالی روی سرور وب اصلی (VM1) ذخیره

جدول ۷: مشخصات خلاصه‌شده رویه‌ها و پورت‌های RPC سیستم

نام رویه	آرگومان ورودی	پاسخ خروجی	پورت شبکه
Login.AuthService	Args.Auth	Reply.Auth	۸۰۸۲
FetchFile.FileService	Args.File	Reply.File	۸۰۸۳

نمی‌شود و تمام فرآیند بارگذاری و واکنشی دارایی‌ها بر عهده ماشین مجازی سوم (VM3) به عنوان سرویس‌دهنده فایل (Server File) می‌باشد. فرآیند هماهنگ دریافت فایل از این گره بک‌آند به صورت یک زنجیره متوالی و بهینه از اتصالات شبکه پیاده‌سازی شده است که گام‌های آن به شرح زیر می‌باشد:

۱. **ارسال درخواست از کلاینت:** کاربر نهایی از طریق واسط کاربری مرورگر خود، درخواست دسترسی به یک فایل یا سند خاص را به پورت ۸۰۸۰ در ماشین VM1 ارسال می‌کند.

۲. **برقراری اتصال بومی:** سرور وب (Web Server) پس از بررسی نشست کاربر، یک اتصال کلیدخورده بر پایه بسته بومی net/rpc به آدرس آی‌پی 192.168.176.131 روی پورت ۸۰۸۳ ایجاد می‌کند.

۳. **فراخوانی رویه از راه دور:** کلاینت مستقر در VM1، نام فایل درخواستی را در ساختار آرگومان ورودی Args.File بسته‌بندی کرده و رویه FetchFile.FileService را در ماشین مقصد فراخوانی می‌کند.

۴. **پردازش دیسک و واکنشی در مقصد:** سرور RPC-Go در ماشین VM3 درخواست را دریافت کرده و مسیر محلی فایل را در دایرکتوری assets/ ارزیابی می‌کند. در صورت وجود فایل، سیستم سیستم‌عامل آن را به صورت بایت‌های خام لود می‌نماید.

۵. **سریال‌سازی باینری بهینه:** داده‌های فایل با استفاده از مکانیزم انکودینگ بومی Gob (تحت عنوان پروتکل Encoding) سریال‌سازی می‌شوند. این روش بدون ایجاد سربارهای متنی، بایت‌های خام فایل را درون ساختار خروجی Reply.File کپسوله‌سازی می‌کند.

۶. **استریم داده روی شبکه:** داده‌ها به صورت یک جریان بایت پیوسته (Stream Byte) روی سوکت شبکه به گره VM1 بازگردانده می‌شوند. سرور وب نیز بدون ذخیره کردن فایل روی دیسک محلی خود، جریان بایت‌های دریافتی را به صورت بوقوع پیوسته (Buffer Chunk) به مرورگر کاربر تحویل می‌دهد تا فرآیند دانلود تکمیل شود.

این مکانیزم توزیع شده (مطابق شکل فرآیندی سیستم) تضمین می‌کند که حافظه مصرفی و دیسک سرور وب اصلی درگیر فایل‌های حجیم نشده و کارایی کل شبکه در بالاترین سطح ممکن (High Throughput) باقی بماند.

۷-۲ توضیح سازوکار الگوی انتشار/اشتراک

به منظور مدیریت ناهمگام هشدارهای سیستم و پایش معیارهای سلامت سرورها بدون ایجاد وابستگی شدید پردازشی میان اجزا، از الگوی توزیع شده انتشار/اشتراک (Pub/Sub Pattern) استفاده شده است. این سازوکار به جای برقراری اتصالات مستقیم و همزمان (Synchronous) میان گره تولیدکننده هشدار و برنامه‌های ناظر، یک لایه واسط به نام کارگزار معرفی می‌کند. این شیوه تفکیک معماری، فرآیند تولید سیگنال‌های خطا را از فرآیند مصرف و نمایش زنده آن‌ها کاملاً مجزا می‌سازد (Loose Coupling).

روال عملیاتی و جریان داده این سازوکار گام به گام به شرح زیر در شبکه توزیع شده پروژه پیاده‌سازی شده است:

۱. **رکن انتشار رویداد (Publisher):** ماشین مجازی اول (VM1) نقش منتشرکننده هشدار را ایفا می‌کند. یک رشته پس‌زمینه پایشگر در سرور وب تعبیه شده است که در فواصل زمانی مشخص (هر ۲ ثانیه یک‌بار)، وضعیت حافظه مصرفی گره را بررسی می‌کند. در صورت وقوع خطا یا شبیه‌سازی نشت حافظه (Memory Leak) و عبور از آستانه مجاز، این ماژول پکت هشدار را ساختاردهی کرده و آن را به صورت ناهمگام (Asynchronous Communication) از طریق پروتکل POST HTTP به پورت ۸۰۸۴ کارگزار ارسال می‌کند.

۲. **رکن کارگزار پیام (Message Broker):** ماشین مجازی چهارم (VM4) به عنوان هسته مرکزی انتقال پیام‌ها در سیستم عمل می‌کند. برنامه‌ی کارگزار به صورت همزمان دو لایه

ارتباطی را مدیریت می‌نماید؛ ابتدا روی پورت ۸۰۸۴ دریافت هشدارهای ورودی از سمت سرور وب می‌ماند و به محض دریافت رویداد جدید، بدون ذخیره‌سازی ایستا، آن هشدار را به صورت لحظه‌ای روی سوکت‌های فعال شبکه (Socket TCP) پورت ۸۰۸۵ برای تمامی مشترکین متصل پخش (Broadcast) می‌کند.

۳. **رکن مشترک (Subscriber):** این بخش شامل برنامه‌های سبک کلاینت در محیط ترمینال است که با بهره‌گیری از فلگ‌های زمان اجرا (Flags Runtime) به آدرس کارگزار متصل می‌شوند. مشترکین یک اتصال پایدار و طولانی‌مدت شبکه را روی پورت ۸۰۸۵ ماشین VM4 برقرار نگه می‌دارند تا به محض انتشار هرگونه سیگنال یا هشدار جدید از سوی کارگزار، داده‌ها را به صورت زنده (Alerting Time-Real) در خروجی خط فرمان به مهندسان سیستم نمایش دهند.

جدول ۸ نقش عملیاتی، نگاشت آدرس‌ها و پروتکل‌های به‌کاررفته در ارکان سه‌گانه این زیرسیستم پایش ناهمگام را نشان می‌دهد.

جدول ۸: مشخصات زیرسیستم جریان رویدادها در الگوی Pattern Pub/Sub

رکن الگو	آدرس آی‌پی	پورت عملیاتی	پروتکل تعاملی
Publisher (VM1)	۱۹۲/۱۶۸/۱۷۶/۱۳۹	۸۰۸۰	POST HTTP
Broker Message (VM4)	۱۹۲/۱۶۸/۱۷۶/۱۳۲	۸۰۸۴/۸۰۸۵	Socket TCP / POST HTTP
Subscriber	۱۹۲/۱۶۸/۱۷۶/۱۳۲	۸۰۸۵	Socket TCP

۲-۸ توضیح روش پایش حافظه

در این سیستم توزیع‌شده (Distributed System)، برای تضمین پایداری گره‌ها و جلوگیری از خرابی‌های ناگهانی ناشی از سرریز منابع، یک مکانیزم خودکار پایش حافظه (Monitoring Memory) در پس‌زمینه ماشین مجازی اول (VM1) پیاده‌سازی شده است. تمرکز اصلی این زیرسیستم بر اندازه‌گیری لحظه‌ای حافظه تخصیص‌یافته روی هپ (Heap Memory) برنامه‌ی در حال اجرا و تشخیص رفتارهای غیرعادی یا نشت حافظه (Memory Leak) است.

روال فنی و گام‌به‌گام روش پایش حافظه به شرح زیر طراحی و اجرا می‌شود:

۱. **نمونه‌برداری دوره‌ای (Periodic Sampling):** پایشگر حافظه با استفاده از یک تیکر پس‌زمینه (Background Ticker) در زبان Go پیکربندی شده است که در فواصل زمانی منظم و پیوسته (هر ۲ ثانیه یک‌بار) فعال می‌شود تا وضعیت مصرف منابع را سنجش کند.

۲. **استخراج شاخص‌های زمان اجرا (Runtime Statistics):** در هر دوره پایش، سیستم با فراخوانی تابع بومی ReadMemStats از بسته runtime، ساختار داده‌ای حاوی جزئیات مصرف حافظه را استخراج می‌کند. متغیر کلیدی مورد بررسی در این ساختار، فیلد Alloc است که مقدار بایت‌های زنده و تخصیص‌یافته روی هپ را که توسط پلتفرم مدیریت حافظه یا جمع‌آوری زباله (Garbage Collection) آزاد نشده‌اند، نشان می‌دهد.

۳. **ارزیابی مرز آستانه (Bound Threshold):** مقدار واکنشی‌شده به طور مداوم با یک حد آستانه از پیش تعیین‌شده مقایسه می‌شود. در سناریوی طراحی‌شده، این آستانه روی مقدار ۳۰۰ مگابایت تنظیم شده است.

۴. **شبیه‌سازی نشت حافظه (Simulation Leak):** به منظور آزمایش عملیاتی زیرسیستم پایش، یک نقطه پایانی اختصاصی برای شبیه‌سازی خرابی تعبیه شده است. با فراخوانی این مسیر توسط توسعه‌دهنده، آرایه‌های بزرگی از داده‌ها به صورت عمدی درون حافظه بارگذاری می‌شوند و بدون آزادسازی باقی می‌مانند تا حجم هپ به سرعت از مرز مجاز عبور کند.

۵. **تولید و ارسال سیگنال هشدار (Alert Generation):** به محض اینکه میزان مصرف حافظه هپ از مرز آستانه فراتر رود، پایشگر بلافاصله یک شیء هشدار شامل مشخصات زمان وقوع، شناسه ماشین و حجم دقیق حافظه مصرفی در قالب ساختار متنی (Pay-load JSON) ایجاد می‌کند. این پکت متنی به صورت ناهمگام از طریق یک درخواست پروتکل ارسال متنی (POST HTTP) به پورت ۸۰۸۴ در ماشین مجازی چهارم (VM4) ارسال می‌گردد تا در لایه کارگزار پیام پردازش شود.

جدول ۹ پیکربندی و پارامترهای عملیاتی این روش پایش را خلاصه می‌کند.

جدول ۹: پارامترهای فنی تنظیم‌شده برای زیرسیستم پایش حافظه در VM1

عنوان پارامتر	مقدار عددی	واحد سنجش	شاخص مورد پایش
بازه نمونه‌برداری	۲	Seconds	Background Ticker
حد آستانه هشدار	۳۰۰	Megabytes	Heap Alloc
پورت مقصد ارسال	۸۰۸۴	Port Network	HTTP POST

۹-۲ سناریوی تست و مستندسازی خروجی‌ها

در این بخش، مراحل اجرای گام به گام سناریوی تست سیستم توزیع شده به همراه خروجی‌های شبیه‌سازی شده در محیط ترمینال و مرورگر مستند شده است.

۱-۹-۲ گام ۱: نمایش آدرس IP ماشین‌های مجازی

برای اطمینان از پیکربندی درست لایه شبکه و تفکیک سرویس‌ها، ابتدا دستور `hostname -I` در هر یک از ماشین‌های مجازی اجرا شده است تا آدرس‌های اختصاصی زیر مانیتور شوند:

مثال ۱.۲. نمایش آدرس شبکه در ماشین مجازی اول (VM1 - Web Server):

```
1 paria@web-vm:~/Documents/web-vm$ hostname -I
2 192.168.176.129
```

مثال ۲.۲. نمایش آدرس شبکه در ماشین مجازی دوم (VM2 - Auth Server):

```
1 paria@auth-vm:~/Documents/auth-vm$ hostname -I
2 192.168.176.130
```

مثال ۳.۲. نمایش آدرس شبکه در ماشین مجازی سوم (VM3 - File Server):

```
1 paria@file-vm:~/Documents/file-vm$ hostname -I
2 192.168.176.131
```

مثال ۴.۲. نمایش آدرس شبکه در ماشین مجازی چهارم (VM4 - PubSub Server):

```
1 paria@pubsub-vm:~/Documents/pubsub-vm$ hostname -I
2 192.168.176.132
```

۲-۹-۲ گام ۲: اجرای سرویس احراز هویت در VM2

سرویس احراز هویت بر پایه‌ی فایل روی ماشین مجازی دوم با دستور `go run main.go` اجرا شده و روی پورت ۸۰۸۲ آمادگی خود را برای دریافت درخواست‌های احراز هویت اعلام می‌کند:

مثال ۵.۲. خروجی ترمینال اجرای سرویس احراز هویت (VM2):

```
1 paria@auth-vm:~/Documents/auth-vm$ go run main.go
2 2026/06/01 17:43:14 Secure JSON-RPC Auth Server active on
   port 8082 (File-Backed)...
```

۲-۹-۳ گام ۳: اجرای سرویس فایل در VM3

در این مرحله، سرویس مدیریت و ذخیره‌سازی فایل‌ها روی ماشین مجازی سوم مستقر شده و با بهره‌گیری از پروتکل بومی Go-RPC روی پورت ۸۰۸۳ فعال می‌شود:

مثال ۶.۲. خروجی ترمینال اجرای سرویس فایل (VM3):

```
1 paria@file-vm:~/Documents/file-vm$ go run main.go
2 2026/06/01 17:44:21 Flexible File Server running on port
   8083 using Native Go-RPC...
```

۲-۹-۴ گام ۴: اجرای سرویس وب در VM1

پس از فعال‌سازی سرویس‌های بک‌اند، سرویس اصلی وب (Web Server) روی گره اول اجرا می‌شود. در زمان راه‌اندازی این سرویس، آدرس‌های IP مربوط به گره‌های دیگر به عنوان فلگ‌های ورودی به برنامه تزریق می‌شوند تا مسیریابی درخواست‌ها به درستی تنظیم گردد:

مثال ۷.۲. خروجی ترمینال راه‌اندازی سرور وب و نگاشت تارگت‌های RPC (VM1):

```
1 paria@web-vm:~/Documents/web-vm$ go run main.go
  -auth-ip=192.168.176.130 -file-ip=192.168.176.131
  -pubsub-ip=192.168.176.132
2 2026/06/01 17:44:52 Initializing Gateway Routing
  Configurations...
3 2026/06/01 17:44:52 -> Auth Server target (VM2):
  192.168.176.130:8082
4 2026/06/01 17:44:52 -> File Server target (VM3):
  192.168.176.131:8083
5 2026/06/01 17:44:52 -> PubSub Server target (VM4):
  192.168.176.132:8084
6 2026/06/01 17:44:52 Web Server running on port 8080...
```

۲-۹-۵ گام پیش‌نیاز: اجرای کارگزار پیام (Broker) در VM4

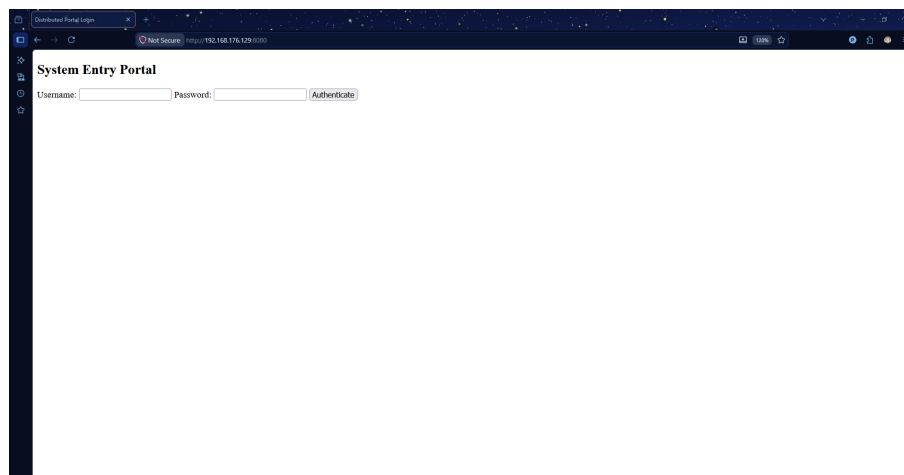
قبل از راه‌اندازی مشترک، موتور اصلی کارگزار پیام (Pub/Sub Broker Engine) روی ماشین مجازی چهارم اجرا می‌شود تا خط‌لوله دریافت داده روی پورت ۸۰۸۴ و ماتریس توزیع پیام روی پورت ۸۰۸۵ آماده به کار شوند:

مثال ۸.۲. خروجی ترمینال راه‌اندازی کارگزار پیام (VM4):

```
1 paria@pubsub-vm:~/Documents/pubsub-vm$ go run broker.go
2 2026/06/01 17:45:07 Pub/Sub Broker Engine initiated on VM
  4...
3 2026/06/01 17:45:07 -> Ingestion webhook pipeline active on
  HTTP port :8084
4 2026/06/01 17:45:07 -> Broadcast routing matrix active on
  TCP port :8085
```

۲-۹-۶ گام ۵: باز کردن صفحه وب از سیستم میزبان

با وارد کردن آدرس آدرس IP سرور وب (VM1) به همراه پورت اختصاصی آن در مرورگر سیستم میزبان، واسط کاربری سامانه با عنوان System Entry Portal بارگذاری شده و آماده دریافت اطلاعات ورودی می‌گردد:



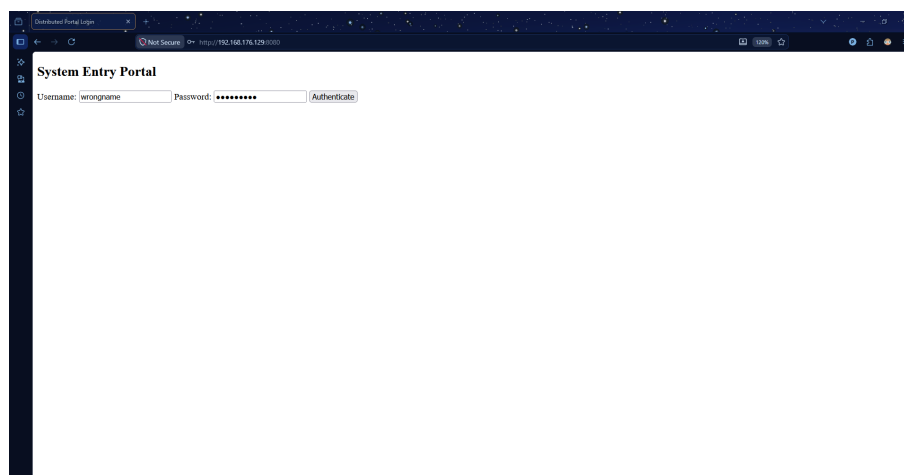
شکل ۱: بارگذاری موفقیت‌آمیز لایه واسط کاربری وب از طریق سیستم میزبان

۲-۹-۷ گام ۶: تلاش برای ورود با اطلاعات نادرست

جهت ارزیابی صحت عملکرد لایه امنیتی، فیلدهای فرم ورود با یک شناسه نامعتبر و تصادفی (مانند نام کاربری wrongname) مقداردهی شده و درخواست با کلیک بر روی دکمه Authenticate ارسال می‌شود:

۲-۹-۸ گام ۷: مشاهده پیام خطای ورود

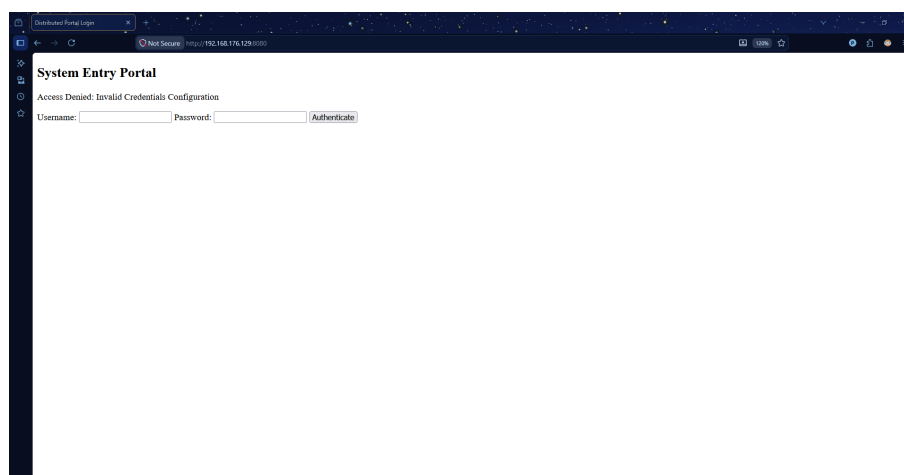
پس از ارسال فرم، دروازه وب درخواست را در قالب یک فراخوانی JSON-RPC به سمت سرور احراز هویت (VM2) هدایت می‌کند. سرویس بک‌آند پس از بررسی پایگاه داده، عدم تطابق مشخصات را احراز کرده و پاسخ منفی صادر می‌کند؛ در نتیجه کلاینت با پیام خطای Access Denied مواجه شده و لاگ مربوطه نیز در ترمینال VM2 با موفقیت ثبت می‌گردد:



شکل ۲: مقداردهی فرم ورود با اطلاعات نامعتبر جهت تست خطای سیستم

مثال ۹.۲. دریافت و پردازش درخواست احراز هویت نامعتبر در ترمینال سرور (VM2):

```
1 paria@auth-vm:~/Documents/auth-vm$ go run main.go
2 2026/06/01 17:43:14 Secure JSON-RPC Auth Server active on
   port 8082 (File-Backed)...
3 2026/06/01 17:46:13 Received JSON-RPC authentication
   request for user: wrongname
```



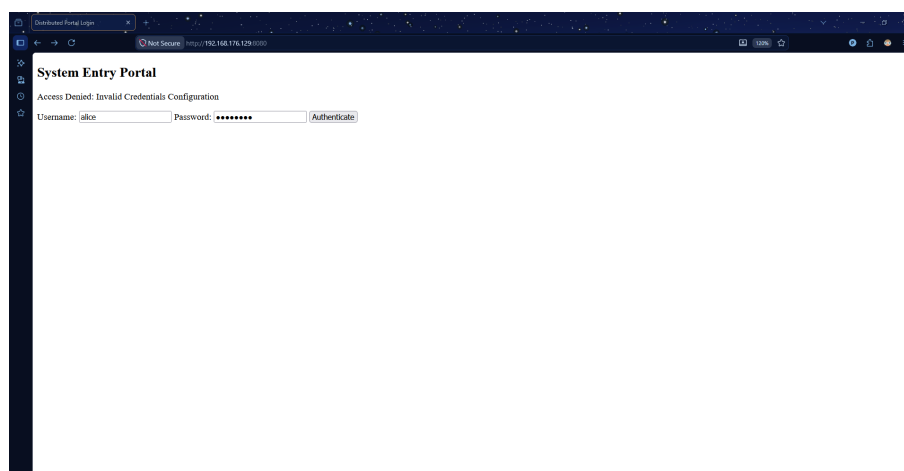
شکل ۳: نمایش پیام خطای عدم دسترسی (Access Denied) در مرورگر کلاینت

۲-۹-۹ گام ۸: ارسال مشخصات معتبر و احراز هویت موفق

در این مرحله، نام کاربری معتبر (مانند alice) در فیلدهای درگاه ورودی وارد شده و درخواست بررسی به سمت بک‌آند ارسال می‌شود. سرور احراز هویت (VM2) درخواست JSON-RPC ارسالی را دریافت کرده و پس از مطابقت دادن اطلاعات با پایگاه‌داده، تاییدیه ورود را صادر می‌کند:

مثال ۱۰.۲. لاگ دریافت و تایید درخواست احراز هویت کاربر alice در ترمینال VM2:

```
1 paria@auth-vm:~/Documents/auth-vm$ go run main.go
2 2026/06/01 17:43:14 Secure JSON-RPC Auth Server active on
   port 8082 (File-Backed)...
3 2026/06/01 17:46:13 Received JSON-RPC authentication
   request for user: wrongname
4 2026/06/01 17:46:56 Received JSON-RPC authentication
   request for user: alice
```



شکل ۴: وارد کردن نام کاربری معتبر alice جهت ورود به سامانه

۲-۹-۱۰ گام ۹: بازیابی خودکار دارایی‌های پویا از سرور فایل (VM3)

بلافاصله پس از تأیید هویت موفق کلاینت، سرور وب (VM1) یک فراخوانی داخلی مبتنی بر پروتکل Native Go-RPC به سرور فایل (VM3) روی پورت ۸۰۸۳ ارسال می‌کند تا تصویر کهکشان

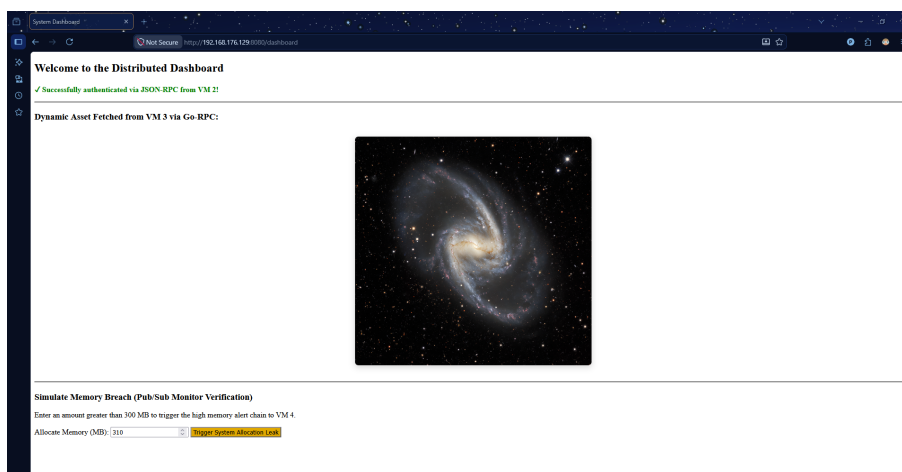
مپ شده را جهت نمایش در داشبورد کاربر بازیابی کند:

مثال ۱۱.۲. لاگ پردازش و ارسال فایل تصویر در ترمینال سرور فایل (VM3):

```
1 paria@file-vm:~/Documents/file-vm$ go run main.go
2 2026/06/01 17:44:21 Flexible File Server running on port
   8083 using Native Go-RPC...
3 2026/06/01 17:46:57 Received request for folder: images,
   file: galaxy.jpeg
```

۲-۹-۱۱ گام ۱۰: بارگذاری کامل داشبورد توزیع شده

پس از تجمیع پاسخ‌های لایه‌های مختلف توزیع شده، کاربر با موفقیت به صفحه‌ی داشبورد مدیریتی (Distributed Dashboard) هدایت می‌شود. در این صفحه، پیام تایید سبز رنگ احراز هویت از VM2 و تصویر بازیابی شده از VM3 به صورت بلادرنگ رندر می‌شوند. همچنین ابزار شبیه‌سازی نشت حافظه جهت ارسال هشدار بالادستی به کارگزار پیام (VM4) در پایین صفحه در دسترس قرار می‌گیرد:



شکل ۵: نمای داشبورد سیستم پس از ورود موفقیت‌آمیز و بارگذاری پویای تصویر کهکشان

۲-۹-۱۲ گام ۱۱: اجرای مشترک (Subscriber) در VM4

پس از آمادگی کارگزار، سرویس مشترک (Subscriber) با مشخص کردن آدرس IP کارگزار اجرا شده و به پورت ۸۰۸۵ متصل می‌گردد تا هشدارهای تلمتری سیستم را به صورت بلادرنگ دریافت کند:

مثال ۱۲.۲. خروجی ترمینال اتصال سرویس مشترک به کارگزار (VM4):

```
1 paria@pubsub-vm:~/Documents/pubsub-vm$ go run subscriber.go
   -broker-ip=192.168.176.132
2 2026/06/01 17:45:15 [+] Establishing subscription link with
   Pub/Sub Broker at 192.168.176.132:8085...
3 2026/06/01 17:45:15 [+] Subscription link active! Listening
   for real-time telemetry alert frames...
```

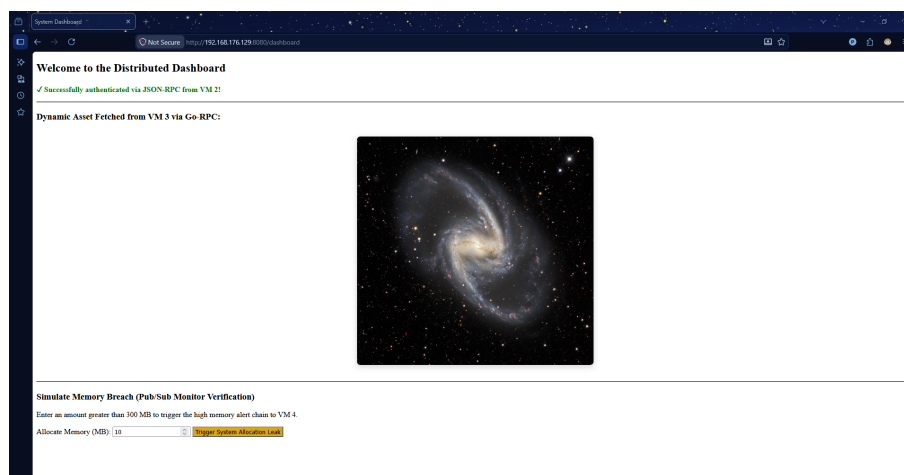
۲-۹-۱۳ گام ۱۲: شبیه‌سازی تخصیص حافظه و پایش وضعیت زیر مرز آستانه

در این گام، لایه پایش سیستم با بررسی میزان مصرف حافظه روی ماشین مجازی اول (VM1) مورد ارزیابی قرار می‌گیرد. با فعال‌سازی فرآیند تخصیص، تغییرات حجم حافظه به صورت لحظه‌ای رصد شده و میزان مصرف کاملاً در محدوده مجاز و زیر مرز آستانه بحرانی (۳۰۰ مگابایت) کنترل می‌شود. هم‌زمان با این فرآیند، گزارش‌های دوره‌ای تلمتری به کارگزار پیام ارسال شده و سرویس مشترک (Subscriber) روی ماشین مجازی چهارم (VM4)، فریم‌های وضعیت پایدار و عادی سیستم را به صورت بلادرنگ دریافت و ثبت می‌نماید:

۲-۹-۱۴ گام ۱۳: شبیه‌سازی نشت حافظه و انتشار هشدار بحرانی بالادستی

در این گام، حجم تخصیص‌یافته حافظه در سرور وب (VM1) از مرز آستانه تعیین‌شده یعنی ۳۰۰ مگابایت عبور داده می‌شود تا مکانیزم اعلان خطر ناهمگام سیستم ارزیابی شود. با تحریک ابزار شبیه‌سازی نشت حافظه و عبور مصرف هپ از مرز مجاز، سیستم پایش داخلی فوراً یک

۲ پیاده‌سازی سیستم توزیع شده چند ماشین مجازی



شکل ۶: شبیه‌سازی تخصیص حجم حافظه در لایه سرور وب (VM1) و پایش وضعیت عادی زیر مرز ۳۰۰ مگابایت



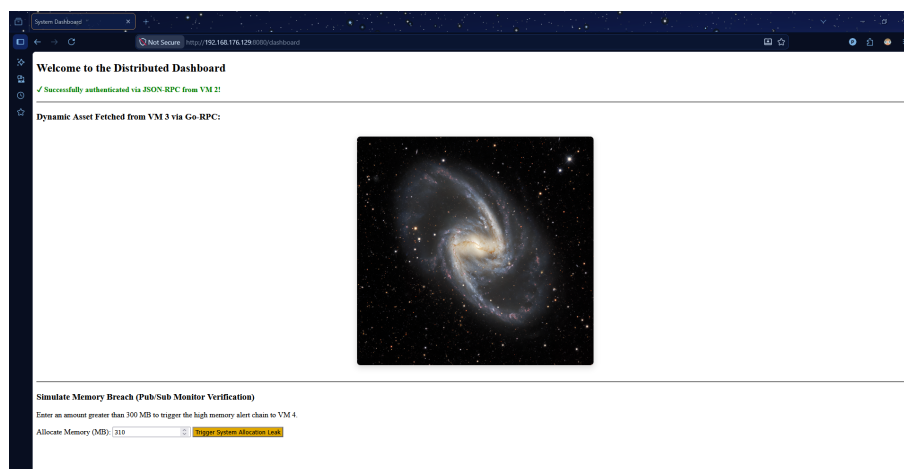
شکل ۷: دریافت و ثبت موفقیت‌آمیز فریم‌های تلمتری وضعیت پایدار توسط سرویس Subscriber روی ماشین چهارم (VM4)

فریم هشدار بحرانی (Critical Telemetry Alert) تولید کرده و آن را به کارگزار پیام (VM4) ارسال می‌کند. کارگزار پیام پس از دریافت داده، ماتریس توزیع را به‌روزرسانی کرده و سیگنال وضعیت اضطراری را به صورت بلادرنگ به سرویس مشترک (Subscriber) ارجاع می‌دهد. مراحل این فرآیند و لاگ‌های ثبت‌شده در گره‌های مختلف شبکه در ادامه مستند شده است:

مثال ۱۳.۲. لاگ‌های ترمینال سرور وب (VM1) در زمان وقوع هشدار و تخصیص حافظه بیش از حد مجاز:

```
1 paria@web-vm:~/Documents/web-vm$ hostname -I
2 192.168.176.129
3 paria@web-vm:~/Documents/web-vm$ go run main.go
   -auth-ip=192.168.176.130 -file-ip=192.168.176.131
   -pubsub-ip=192.168.176.132
4 2026/06/01 17:44:52 Initializing Gateway Routing
```

۲ پیاده‌سازی سیستم توزیع شده چند ماشین مجازی



شکل ۸: نمای واسط کاربری داشبورد پس از تحریک ابزار شبیه‌سازی نشت حافظه و عبور از مرز ۳۰۰ مگابایت



شکل ۹: ثبت لاگ فرآیند تخصیص پله‌پله حافظه و فعال شدن آستانه بحرانی در ترمینال سرور (VM1) وب

```
Configurations...
5 2026/06/01 17:44:52 -> Auth Server target (VM2):
    192.168.176.130:8082
6 2026/06/01 17:44:52 -> File Server target (VM3):
    192.168.176.131:8083
7 2026/06/01 17:44:52 -> PubSub Server target (VM4):
    192.168.176.132:8084
8 2026/06/01 17:44:52 Web Server running on port 8080...
9 2026/06/01 17:49:12 ALERT: Memory utilization breached
    threshold! Current: 330 MB
10 2026/06/01 17:49:14 ALERT: Memory utilization breached
    threshold! Current: 330 MB
11 2026/06/01 17:49:16 ALERT: Memory utilization breached
```



```

threshold! Current: 330 MB
12 2026/06/01 17:49:18 ALERT: Memory utilization breached
threshold! Current: 330 MB
13 2026/06/01 17:49:20 ALERT: Memory utilization breached
threshold! Current: 330 MB

```

مثال ۱۴.۲. دریافت و مسیریابی فریم‌های تلمتری وضعیت اضطراری توسط کارگزار پیام (Pub/-) روی (Sub Broker (VM4):

```

1 paria@pubsub-vm:~/Documents/pubsub-vm$ hostname -I
2 192.168.176.132
3 paria@pubsub-vm:~/Documents/pubsub-vm$ go run broker.go
4 2026/06/01 17:45:07 Pub/Sub Broker Engine initiated on VM
   4...
5 2026/06/01 17:45:07 -> Ingestion webhook pipeline active on
   HTTP port :8084
6 2026/06/01 17:45:07 -> Broadcast routing matrix active on
   TCP port :8085
7 2026/06/01 17:45:15 Subscriber connected successfully from
   network node: 192.168.176.132:53312
8 2026/06/01 17:49:12 Telemetry Alert received from Web
   Server! Fanout broadcasting to active listeners...
9 2026/06/01 17:49:14 Telemetry Alert received from Web
   Server! Fanout broadcasting to active listeners...
10 2026/06/01 17:49:16 Telemetry Alert received from Web
   Server! Fanout broadcasting to active listeners...
11 2026/06/01 17:49:18 Telemetry Alert received from Web
   Server! Fanout broadcasting to active listeners...

```

```
12 2026/06/01 17:49:20 Telemetry Alert received from Web
    Server! Fanout broadcasting to active listeners...
```

مثال ۱۵.۲. دریافت آئی رویداد خطا و پارس ساختار جی‌سان (JSON) توسط سرویس مشترک
:(Subscriber)

```
1 paria@pubsub-vm:~/Documents/pubsub-vm$ go run subscriber.go
  -broker-ip=192.168.176.132
2 2026/06/01 17:45:15 [+] Establishing subscription link with
  Pub/Sub Broker at 192.168.176.132:8085...
3 2026/06/01 17:45:15 [+] Subscription link active! Listening
  for real-time telemetry alert frames...
4
5 [SYSTEM WARN] TELEMETRY EVENT INGESTED:
6 {"event_type": "HIGH_MEMORY_USAGE", "service": "web-server",
7
8 "memory_mb": 330, "threshold_mb": 300,
9
10 "timestamp": "2026-06-01T17:49:12.898641332+03:30"}
11
12 [SYSTEM WARN] TELEMETRY EVENT INGESTED:
13 {"event_type": "HIGH_MEMORY_USAGE", "service": "web-server",
14
15 "memory_mb": 330, "threshold_mb": 300,
16
17 "timestamp": "2026-06-01T17:49:14.898489561+03:30"}
18
19 [SYSTEM WARN] TELEMETRY EVENT INGESTED:
```

```

20 {"event_type": "HIGH_MEMORY_USAGE", "service": "web-server",
21
22 "memory_mb": 330, "threshold_mb": 300,
23
24 "timestamp": "2026-06-01T17:49:16.898669833+03:30"}
25
26 [SYSTEM WARN] TELEMETRY EVENT INGESTED:
27 {"event_type": "HIGH_MEMORY_USAGE", "service": "web-server",
28
29 "memory_mb": 330, "threshold_mb": 300,
30
31 "timestamp": "2026-06-01T17:49:18.900398999+03:30"}

```

۲-۱۰ مشکلات و محدودیت‌های پیاده‌سازی

با وجود پیاده‌سازی موفقیت‌آمیز و جداسازی وظایف گره‌ها در این مخزن همه‌منظوره، معماری فعلی سیستم توزیع‌شده (Distributed System) با چالش‌ها، محدودیت‌ها و نقاط ضعفی روبرو است که استقرار آن در محیط‌های عملیاتی واقعی را نیازمند بازنگری و بهبودهای ساختاری می‌کند. مهم‌ترین محدودیت‌های شناسایی‌شده در جریان توسعه سیستم به شرح زیر می‌باشند:

۱. **نقطه شکست واحد (Single Point of Failure):** ماشین مجازی اول (Web Server) به عنوان دروازه ورود اصلی به کل سیستم عمل می‌کند. در صورت بروز هرگونه خرابی نرم‌افزاری یا قطع اتصال در این گره، کل سیستم به طور کامل از دسترس خارج شده و کاربران امکان احراز هویت یا دریافت فایل را نخواهند داشت. همچنین لایه کارگزار پیام (Message Broker) در ماشین VM4 نیز یک نقطه شکست واحد دیگر برای لایه پایش محسوب می‌شود.

۲. **عدم پایداری پیام‌ها در کارگزار (Lack of Message Persistence):** زیرسیستم انتشار/اشتراک

در ماشین مجازی چهارم هشدارهای دریافتی را به صورت کاملاً گذرا و فرار پخش می‌کند. در صورتی که یک کلاینت ناظر (Subscriber) در زمان ارسال هشدار نشت حافظه قطع باشد، هیچ مکانیزمی برای ذخیره‌سازی ایستا و بازیابی پیام‌های از دست رفته وجود ندارد.

۳. **فقدان امنیت و رمزنگاری ترافیک (Lack of Network Security):** تمامی ارتباطات ساختاریافته

میان ماشین‌های مجازی، از جمله فرستادن اطلاعات ورود کاربران در پروتکل JSON-RPC و استریم بایت‌های خام فایل‌ها در پروتکل Go-RPC، بر بستر سوکت‌های شبکه خام انجام می‌شود و فاقد لایه‌های رمزنگاری (مانند TLS) است که سیستم را در برابر حملات شنود آسیب‌پذیر می‌سازد.

۴. **عدم همگام‌سازی و تکثیر داده‌ها (Lack of Data Consistency):** لایه ذخیره‌سازی

دارایی‌های دیجیتال در ماشین مجازی سوم به صورت متمرکز است. در این سیستم هیچ‌گونه مکانیزمی برای نسخه‌برداری پشتیبان (Data Replication) یا توزیع فایل‌ها روی گره‌های موازی تعبیه نشده است که ریسک نابودی دائمی داده‌ها را در صورت بروز خطای دیسک در VM3 بالا می‌برد.